

FinSight_多智能体金融研报热点专项面试题库

定位：面向大模型应用开发、Agent 工程、RAG/Evaluation 方向面试的项目题库。
使用方式：先熟练核心 20 问，再按岗位侧重点复习 Trade-off 与专项场景题。
项目事实来源：仅限当前 DeepReport_plus 仓库中的代码、配置、测试与正式评测产物。

0. 事实边界与简历表述口径

0.1 项目命名说明

- 简历与面试表述统一使用 **FinSight_多智能体**。
- 仓库目录、历史 README 与部分运行时字符串仍保留 DeepReport_plus 、 Open DeepReport++ 或 DeepReportPlusMCP ；这属于工程内部既有标识，本题库不声称已完成代码级全面改名。

0.2 当前仓库已经证明的能力

能力	可安全陈述的事实	证据位置
多智能体研报链路	可编排检索、证据规范化、分析/估值、写作、校验与修复，并产出结构化运行记录	src/agents/multi_agent_orchestrator.py
可追溯产物	运行可输出 evidence.json 、 claims.json 、 citations.json 、 估值产物、校验报告与 trace	src/agents/multi_agent_orchestrator.py
正式方案比较	在冻结 formal18_fy2024_v1 快照上比较三种 variant	docs/formal_benchmark_protocol.md 、 eval_outputs/benchmark_formal18_fy20
工具调用	核心工具注册表暴露 9 个真实英文 API	src/tools/registry.py
Skill	支持 YAML 配置、选择并向 Planner/Router 渲染能力提示	src/tools/skill_registry.py 、 configs/skill_registry.yaml
MCP-style	支持本地工具发现、schema 暴露、调用、manifest 导出，以及 HTTP JSON-RPC tools 表面	src/utils/mcp_manager.py 、 src/utils/mcp_http_server.py
Durable Memory	有持久上下文实现与测试，但默认关闭，事实仍必须来自当前证据	src/agents/durable_memory.py 、 configs/app.yaml

0.3 必须限定或禁止的说法

不能直接说	面试中应改成
“完整落地了生产级 MCP 平台”	“落地了 local-mcp-v1 的 MCP-style tools 边界和 HTTP JSON-RPC 调用表面，resources/prompts 仍是空集合。”
“Skill 是已执行的复杂子流程平台”	“ SkillRegistry 当前负责 Planner/Router 的能力提示与选择，工具实际执行仍由编排链路承担。”
“Memory 会提供历史事实”	“durable memory 默认关闭，即使启用也只提供上下文提示，不能替代 evidence_id 引用。”
“评测说明系统可用于投资决策”	“评测证明其在冻结证据协议下的报告交付与可追溯性表现，不证明投资准确率或线上稳定性。”
“项目已经全部改名 FinSight_多智能体”	“FinSight_多智能体是简历展示名，仓库仍保留历史工程标识。”

0.4 正式评测结果速记

冻结协议： formal18_fy2024_v1 ，覆盖 US 6 、 HK 6 、 CN-A 6 个 FY2024 case ；所有 variant 使用相同冻结证据快照，正式 runner 禁止运行时抓取。

Variant	交付通过率	客观质量分	关键结论可追溯率 v1
direct_llm	16.67%	51.21	29.66%
single_agent_rag	27.78%	52.52	34.89%
multi_agent_rag	72.22%	86.27	70.01%

边界必须一起说：multi_agent_rag 在该冻结协议上优于两个 one-shot baseline；但 HK 可追溯率仅 37.80%，CN-A 交付通过率仅 50.00%，结果不能外推成生产稳定性或投资判断准确率。

0.5 工具与接口中文展示表

面试中不要只背英文函数名，应说成“中文功能名（英文 API）：用途注释”；英文标识保持与代码完全一致。

中文功能名（英文 API）	用途注释
检索本地证据（retrieve_local_evidence）	按查询、标的和期间检索本地证据，支持 bm25、vector、hybrid、reranker、hybrid_rerank 排序模式。
计算财务比率（calculate_financial_ratios）	从证据记录抽取收入、利润率、ROE/ROA 与现金流特征。
构建趋势特征（build_trend_features）	汇总证据覆盖和趋势特征，供分析与报告引用。
构建三表视图（build_three_statement_view）	将可用证据规范化为损益、现金流和资产负债表行项目。
构建同行比较（build_peer_comparison）	生成同行表格和目标公司排序语境。
执行公司估值（perform_company_valuation）	输出 P/E、P/S、DCF 与同行语境下的初步估值及相关输入。
获取市场快照（fetch_yahoo_market_snapshot）	获取行情快照并转换为可被引用的证据输入。
渲染全部图表（render_all_charts）	根据特征文件生成图表及元数据。
挂载图表到报告（attach_charts_to_report）	将图表元数据挂接到已有 Markdown 报告。

接口边界：MCP manifest 中 qualified name 形如 finance.retrieve_local_evidence；暴露为 Function Calling schema 时形如 finance__retrieve_local_evidence。当前 HTTP JSON-RPC 入口支持 tools 调用，resources/list 与 prompts/list 返回空集合。

0.6 证据产物速查

叙述点	可展示的实际产物
证据进入系统	evidence.json
关键结论结构化	claims.json、citations.json
数字与估值可复算	financial_metrics.json、valuation_model.json、valuation_assumptions.json、valuation_sensitivity.json
图表一致性	charts.json、chart_consistency.json、multimodal_consistency.json
门禁与修复	verification_report.json、company_report_scorecard.json、gap_resolution_trace.jsonl
Agent 与工具观测	task_trace.jsonl、agent_collaboration_trace.json、tool_trace.json、mcp_manifest.json
最终交付	report.md、report.html、report.json

第一部分：核心 20 问双层答案

Q1. 请介绍 FinSight_多智能体项目

30 秒简答

FinSight_多智能体是一个证据驱动的金融研报生成系统。输入是标的、期间、报告需求和关注主题，系统把检索、财务分析、估值、写作、引用校验与缺口修复拆成可追溯步骤，输出 Markdown/HTML/JSON 报告以及 `claims`、`citations`、`verification` 和 `trace` 产物。它的重点不是让模型自由写作，而是让关键结论有来源、数字可复算、失败可定位。

详细答案

我把研报生成处理为 `claim-first` 工作流：Planner 组织研究任务，检索与规范化阶段生成证据，分析阶段从证据和工具结果形成 `claim` 与估值材料，写作阶段只基于这些中间产物组织报告，Verifier 与修复流程检查未支撑结论、引用缺口和一致性问题。最终除了报告文本，还保留每一步的结构化依据，方便回归评测和排错。项目还用冻结 `Formal-18` 对比了 Direct LLM、Single-Agent RAG 和 Multi-Agent RAG，验证了这种拆解在固定协议下对可交付性和可追溯性的收益。

项目依据

- `src/agents/multi_agent_orchestrator.py`：编排、产物写出与修复记录。
- `docs/formal_benchmark_protocol.md`：冻结输入与三方案比较协议。
- `eval_outputs/benchmark_formal18_fy2024_v1/formal_benchmark_report.md`：正式结果。

表述边界与 Trade-off

可说“面向研报草稿和质量控制”，不要说“自动投资决策系统”。分阶段与校验提高可追溯性，但会增加工具调用、时延和工程复杂度，因此简单低风险摘要未必需要完整链路。

Q2. 为什么金融研报任务适合 Multi-Agent?

30 秒简答

金融研报不是单纯写作，而是证据检索、数字计算、估值假设、图表、风险披露和引用校验的组合任务。Multi-Agent 的价值是把不同责任拆成可检查产物；在本项目冻结 `Formal-18` 上，`multi_agent_rag` 的交付通过率和可追溯率明显高于两个 `one-shot baseline`。

详细答案

单次生成容易出现三类问题：事实没有证据、数字和图表不可复算、错误只能看到最终文本而找不到责任步骤。FinSight_多智能体将证据、`claims`、估值输入、引用和校验拆开，让失败能够回到检索、分析或写作环节修复。固定 FY2024 快照比较中，`multi_agent_rag` 达到 72.22% 交付通过率、86.27 客观质量分、70.01% 可追溯率，而 `single_agent_rag` 分别为 27.78%、52.52、34.89%，说明在该协议上，多阶段约束比单次带检索写作更可靠。

项目依据

- `configs/benchmark_formal18_fy2024.yaml`：三个 variant 和 18 个固定 case。
- `eval_outputs/benchmark_formal18_fy2024_v1/formal_benchmark_report.md`：结果与市场弱项。

表述边界与 Trade-off

这不是“Agent 越多越好”。多 Agent 牺牲延迟、成本和维护难度，适合带财务结论、估值或合规风险的报告；对仅需快速概览的请求，可选更短链路并标注输出等级。

Q3. 每个 Agent 的职责与边界是什么？

30 秒简答

我把职责按产物拆分：规划负责任务分解，检索与浏览负责证据，分析负责财务 `claim` 和估值输入，写作负责报告组织，校验负责发现证据与一致性缺口，修复只针对失败项回补。数字尽量来自工具，不让 Writer 凭语言能力重新计算。

详细答案

一个可靠划分原则是“谁产生数据，谁对该数据结构负责”。检索阶段产出 `evidence.json`；分析阶段形成 `claims.json`、财务指标和估值产物；写作阶段引用既有 `claim` 与 `citation`；校验阶段输出 `verification_report.json` 和 `scorecard`；有缺口时再路由到相应环节，而不是整篇重写。这样报告中某个数字失败时，可以追到证据、分析工具或写作引用，不必猜模型哪里发挥错了。

项目依据

- `src/agents/multi_agent_orchestrator.py`：分析、写作、验证、rework 与 `collaboration trace`。
- `src/tools/registry.py`：财务计算、估值、图表工具的职责。

表述边界与 Trade-off

职责拆得太细会带来重复上下文和编排成本；拆得太粗又失去错误定位能力。本项目的面试表述应围绕“证据、计算、成文、校验”四类可验证责任，而不是虚构更多自治角色。

Q4. 共享状态和结构化中间产物怎么设计？

30 秒简答

共享状态不能只是聊天历史，至少要保存任务计划、evidence、claims、估值假设、citations、图表一致性和 verification 结果。每个 Agent 对自己的字段写入，后续节点读取并校验，这样可以重放、审计和局部修复。

详细答案

研报任务需要把“事实来源”和“文字输出”解耦。证据记录提供来源和期间，`claim` 绑定 `evidence id` 与数值 `lineage`，估值产物保留模型和敏感性输入，引用与图表记录最终使用位置，Verifier 根据这些结构化信息判断是否可交付。FinSight_多智能体的输出目录就是这种 `blackboard` 的外显版本：不仅有最终报告，还有 `task_plan`、`evidence`、`claims`、`citations`、估值和 `trace` 文件。

项目依据

- `src/agents/multi_agent_orchestrator.py`：输出目录中的结构化 `artifact` 清单。
- `src/evaluation/benchmark_metrics.py`：通过结构化产物计算交付和追溯指标。

表述边界与 Trade-off

状态越丰富，审计越强，但 `schema` 版本治理和上下文裁剪成本也越高。应把高风险事实和 `lineage` 放在结构化状态中，把长文本仅按需要注入模型上下文。

Q5. Evidence Record 如何设计？

30 秒简答

Evidence Record 要解决“这句话依据哪份资料、对应哪个标的和期间、能否被关键结论引用”的问题，因此至少包含 `evidence_id`、来源、标题或内容、标的、期间、时间与元数据。关键数字 `claim` 不能只引用一段模糊文本，还要能关联具体数值来源。

详细答案

对金融任务，来源优先级、财年口径和时间都影响结论可信度。检索回来的内容应先规范化为 `evidence`，再由分析节点形成 `claim`；报告写作引用 `claim` 时保留 `evidence_id`。冻结评测进一步要求关键 `claim` 的引用必须存在于该 `case` 的快照中，数字 `claim` 还需通过 `linked-evidence numeric audit`，从而阻止“引用存在但数字没有支持”的假追溯。

项目依据

- `src/agents/multi_agent_orchestrator.py`：`evidence.json`、`claims.json`、`citations.json`。
- `docs/formal_benchmark_protocol.md`：关键 `claim` 的追溯计分规则。
- `tests/test_formal_benchmark.py`：快照引用与数字审计测试。

表述边界与 Trade-off

证据颗粒度太粗不利于数字定位，太细会增加索引和上下文成本。面试可说“按段落/表格/指标保留足够 `lineage`”，不能说任何引用都天然等于事实正确。

Q6. 金融 RAG 的切片策略怎么设计？

30 秒简答

金融材料不能只按固定字符切片，应优先保留章节、表格和指标语义边界：业务说明按段落，财务表按表头与行项目，风险和估值假设按完整语段。检索片段还必须携带来源、标的、期间和单位元数据。

详细答案

年报中正文、三表和脚注的召回需求不同：正文适合段落检索，财务数字需要表格或指标级片段，估值与风险需要上下文完整性。实践中应保留 `symbol`、`period`、`source_type`、单位与原始 `evidence` 关联，避免 `Writer` 将跨财年或跨币种的片段拼在一起。项目工具的检索接口已支持 `use_chunks`，说明回答应落到片段化检索和后续 `lineage`，而不是只背通用 `chunk size`。

项目依据

- `src/tools/registry.py`：检索工具的 `use_chunks` 与 `ranking_mode` 参数。
- `src/agents/multi_agent_orchestrator.py`：PDF sections/tables 转为 `evidence record` 的路径。

表述边界与 Trade-off

结构化切片提高可复算与精确引用，但解析和索引复杂度更高；对于短的纯文本披露，可使用更简单段落切片，不必强行表格化。

Q7. Hybrid Retrieval 和 Rerank 怎么取舍？

30 秒简答

本项目检索入口真实支持 `bm25`、`vector`、`hybrid`、`reranker` 和 `hybrid_rerank`。我会用 `bm25` 做低成本、强关键词基线；语义表述变化较大时引入向量或混合召回；只有对关键章节或高风险结论，才承担 `rerank` 的延迟换更好的排序质量。

详细答案

`BM25` 对股票代码、指标名和明确关键词稳定且可解释；`vector` 更适合语义近似，但可能召回概念相关却口径不符的内容；`hybrid` 结合两者以降低单路漏召；`reranker` 或 `hybrid_rerank` 对候选集重排，适合提高前排证据质量。当前实现中向量搜索失败会回退到 `BM25-only`，`reranker` 使用 `checkpoint` 路径，因而回答要包含回退和依赖治理。评测时不能只看生成文本，还应比较召回质量、引用支撑率、`p95` 延迟与成本。

项目依据

- `src/retrieval/retrieve.py`：五种 `mode`、融合排序和向量失败回退。
- `src/tools/registry.py`：对 `Agent` 暴露的检索参数 `schema`。

表述边界与 Trade-off

不能直接说 `hybrid_rerank` 一定最好：它增加模型/索引依赖、延迟和调参成本。安全决策是按任务风险分层，并用 `Harness` 验证质量收益是否覆盖资源代价。

Q8. 财务比率和估值由 LLM 算还是工具算？

30 秒简答

数字计算尽量交给确定性工具，LLM 负责解释与组织结论。本项目有财务比率、三表视图、同行比较与公司估值工具，并保留估值模型、假设和敏感性产物，便于校验和复算。

详细答案

如果让 LLM 在自然语言中直接完成单位转换、比率和估值，错误不易定位，也无法稳定复现。`FinSight` 多智能体将证据记录传给工具形成指标和估值 `artifacts`，再让写作节点基于产物生成段落；`Verifier` 检查关键结论是否绑定证据和数值 `lineage`。估值特别需要把假设和敏感性分析保存出来，否则即使目标价看似合理，也不能作为可交付结论。

项目依据

- `src/tools/registry.py`：`calculate_financial_ratios`、`build_three_statement_view`、`build_peer_comparison`、`perform_company_valuation`。
- `src/agents/multi_agent_orchestrator.py`：估值产物写出。

表述边界与 Trade-off

工具化提高确定性和审计性，但输入数据缺失或口径不齐时也会失败。应披露缺口或降级结论，不用 LLM 猜一个完整估值。

Q9. Quality Gate 检查哪些维度？

30 秒简答

质量门禁至少检查：关键结论是否有证据、数字是否有 lineage、图表与正文是否一致、估值是否可复算、必要章节与风险披露是否存在。门禁不是追求文笔分数，而是阻止高风险错误报告被当成可交付稿件。

详细答案

FinSight_多智能体会写出 verification_report.json、company_report_scorecard.json、chart_consistency.json 与 multimodal_consistency.json 等产物。正式评测把交付通过率、客观质量分和关键结论可追溯率分开报告，因为一篇报告可能语言流畅却引用不可靠，也可能证据完整但表达较弱。阻断项应集中在无支撑关键事实、严重数字/图表冲突和缺失必要风险披露；低风险表述问题可以 warning 或进入修复。

项目依据

- src/agents/multi_agent_orchestrator.py：verification、scorecard、一致性产物。
- src/evaluation/benchmark_metrics.py 与测试：交付与追溯规则。

表述边界与 Trade-off

门禁越严格，误交付越少，但拒绝率和生成耗时会上升。应将“不可交付”和“带缺口披露的研究草稿”区分，而不是为了通过率放松关键事实门槛。

Q10. 生成、校验、修复闭环怎么实现？

30 秒简答

先基于 evidence 和结构化分析产出草稿，再由 Verifier 标记具体缺口，最后只把失败问题路由回对应环节修复，并记录修复 trace 与轮次上限。这样比整篇无约束重写更稳定，也更容易审计。

详细答案

闭环的关键不是“让模型再写一次”，而是让验证结果成为明确输入：例如缺关键引用回到证据或引用组装，估值假设不全回到分析节点，图文冲突回到图表或正文生成。FinSight_多智能体保存 verification、gap resolution、revision history 和 agent/tool trace，便于确认修复是否带来了新问题。循环必须有最大轮数、预算和无法修复时的降级规则，否则 Agent 可能持续消耗资源却无法提高质量。

项目依据

- src/agents/multi_agent_orchestrator.py：_run_quality_rework_loop 相关逻辑及 gap_resolution_trace、revision_history 产物。
- tests/test_benchmark_metrics.py：门禁失败原因的可检查性。

表述边界与 Trade-off

局部修复通常降低副作用，但复杂跨章节冲突可能需要整体重组。面试应说明终止条件和披露策略，不应承诺所有失败都能自动修好。

Q11. Benchmark Harness 是什么？为什么不是简单评测脚本？

30 秒简答

Benchmark Harness 是用于公平跑方案、收集产物、计算指标和定位失败的一套评测执行框架。对 FinSight_多智能体来说，它不只是调用一个评分函数，而是用同一冻结证据集运行 Direct LLM、Single-Agent RAG、Multi-Agent RAG，再输出总体、分市场和失败类型结果。

详细答案

报告系统的比较必须控制输入、快照与评价协议，否则“新方案更好”可能只是抓取了更新资料或用了不同数据。正式 Harness 将网络采集和评测 runner 隔离：先将 FY2024 资料冻结并校验 hash，再让三个 variant 使用同一个快照，最后统计交付通过率、客观质量分与关键结论追溯率。这样不仅能给出总分，还能发现例如港股引用弱或 A 股交付失败这样的结构性问题。

项目依据

- docs/formal_benchmark_protocol.md：快照、variant、公平性与指标协议。
- src/evaluation/formal_benchmark.py：正式 runner 和报告输出。
- tests/test_formal_benchmark.py：不完整快照拒跑与同快照运行三方案。

表述边界与 Trade-off

冻结 Harness 强于可复现比较，但不能衡量当天新闻 freshness；线上表现需要另建实时评估，不应把离线结果冒充实时生产表现。

Q12. Formal-18 冻结评测集怎么设计？

30 秒简答

Formal-18 是 18 个跨市场 FY2024 标的组成的冻结回归集，分别覆盖美股、港股和 A 股各 6 个 case。它的目的不是统计代表整个市场，而是在一致、可复验的证据快照上比较系统方案并暴露失败类型。

详细答案

项目先通过独立 staging 流程准备公开来源数据，再冻结为 formal18_fy2024_v1 快照并记录 hash；正式 runner 不允许在线抓取。三种方案面对相同 case 与证据池，关键 claim 的可追溯计算还要求引用存在于该快照、数字能够通过 audit。18 条规模适合工程回归与基线比较，但不足以说明跨年度、跨事件或所有行业都稳定，因此仍需要后续扩展集和隐藏集。

项目依据

- configs/benchmark_formal18_fy2024.yaml：18 个 case 与市场分布。
- docs/formal_benchmark_protocol.md：staging、snapshot 和计分定义。

表述边界与 Trade-off

冻结样本确保复现性，却以新鲜度和覆盖广度为代价。应说“formal regression benchmark”，不要称为行业通用 benchmark 或线上成功率。

Q13. 三个 baseline 怎么公平比较？结果怎么解释？

30 秒简答

公平比较的前提是同一 case、同一冻结证据池、明确的执行约束和同一指标口径。本项目结果显示 multi_agent_rag 在冻结协议下显著优于 direct_llm 与 single_agent_rag，但结论必须绑定数据集版本和已知市场弱项。

详细答案

三个 variant 的区别在于处理方式：direct_llm 基于冻结证据做一次生成，single_agent_rag 先用本地 BM25 选证据再一次生成，multi_agent_rag 走当前编排链和校验/修复。在一致协议下，三者交付通过率为 16.67%、27.78%、72.22%，追溯率为 29.66%、34.89%、70.01%。这支持“结构化编排在该任务集上更强”的结论；同时多 Agent 仍发生 5 个交付失败，且港股追溯明显不足，所以不能只展示最好看的总指标。

项目依据

- docs/formal_benchmark_protocol.md：variant 定义及隔离要求。
- eval_outputs/benchmark_formal18_fy2024_v1/formal_benchmark_report.md：总体、市场与失败结果。

表述边界与 Trade-off

Multi-Agent 的质量收益附带更高运行复杂度和可能更高延迟/费用；是否用于每次请求需根据报告风险、交付时限和资源预算决定。

Q14. 三个核心评测指标怎么定义？

30 秒简答

交付通过率回答“这份报告能否按规则交付”；客观质量分回答“结构、数字和质量约束表现如何”；关键结论可追溯率回答“高风险结论是否真的有冻结证据和数字 lineage 支持”。三者必须同时看，不能用语言质量掩盖证据失败。

详细答案

交付通过率由确定性 delivery checks 约束，适合做是否出稿的底线指标。客观质量分聚合报告质量检查，但不等价于关键事实都可靠。Traceable Claim Rate v1 只计入明确标注为 critical 且类型许可的 claim，并要求引用存在、来自该 case 快照，数值 claim 通过 numeric audit。其宏平均防止某些样本生成大量简单 claim 拉高整体表现，微观比例只作诊断补充。

项目依据

- docs/formal_benchmark_protocol.md：traceable claim 规则与 macro/micro 口径。
- src/evaluation/benchmark_metrics.py、tests/test_benchmark_metrics.py：门禁和 lineage 验证行为。

表述边界与 Trade-off

指标设计越严谨越能防止虚高分，但也会漏掉主观分析质量或投资价值判断。因此本项目将目标限制为报告工程质量和可追溯性，不评价收益预测。

Q15. Skill 在系统里是什么？

30 秒简答

在当前 FinSight_多智能体实现中，Skill 不是独立执行器，而是提供给 Planner/Router 的可配置能力提示。它描述某类任务适用哪些工具、输入输出和 guardrails，帮助路由选择；真正的工具调用、证据和校验仍由编排链负责。

详细答案

SkillSpec 包含名称、描述、适用 agent types、触发词、工具名、输入输出摘要和 guardrails。当前配置中有证据发现、财务分析、报告组装、校验修复、行业研究与宏观上下文等条目。系统按 query 和 task type 选择相关 Skill 并生成 compact brief 注入 Planner/Router；brief 本身明确写着 skills 不替代工具执行、证据、引用或 verifier gates。这与“一个 Skill 自主执行多工具并管理重试”的更完整平台仍有距离。

项目依据

- src/tools/skill_registry.py：SkillSpec、select、render_brief 及明确边界文本。
- configs/skill_registry.yaml：实际配置项。
- tests/test_skill_registry.py：选择与 YAML 加载测试。

表述边界与 Trade-off

静态能力提示轻量、可控、便于配置，但执行能力有限；升级为可执行 Skill 会增强复用，也会新增版本、错误恢复和权限治理成本。

Q16. Skill Registry 与路由怎么设计和评估？

30 秒简答

当前 Registry 用 YAML 管理能力元数据，路由按任务类型和触发词选择最相关的 Skill brief 给 Planner/Router。评估重点不是炫耀自动化，而是看正确能力是否被提示、是否减少遗漏，同时确保它不能越过 evidence 和 verifier 边界。

详细答案

配置化的优势是不改编排代码就调整任务提示、工具集合和 guardrails。当前选择逻辑按 agent_types 和 query 触发词打分，适合轻量可解释路由；可用针对典型请求的单测或 Harness case 检查应选 Skill 是否排前。对于利润弹性或估值等高风险需求，即使命中了分析 Skill，也必须让工具输出数值材料并接受引用/估值门禁，而不是将路由命中视为完成任务。

项目依据

- src/tools/skill_registry.py 与 configs/skill_registry.yaml：规则型选择与渲染。
- tests/test_skill_registry.py：financial_statement_analysis 等选择测试。

表述边界与 Trade-off

规则路由透明且低成本，但对复杂表达泛化有限；模型路由可能提升覆盖，却带来不可解释选择和额外评测成本。当前安全表述是“实现可配置提示路由，并可逐步评估扩展”。

Q17. MCP 在项目中解决什么问题？

30 秒简答

当前项目落地的是本地 MCP-style 工具边界：把已有 9 个金融工具以 `finance.*` 名称发现、导出 schema、调用并写出 manifest，同时提供 HTTP JSON-RPC 的 `tools/list` 和 `tools/call` 表面。它让工具接口更统一，但还不是完整 MCP 平台。

详细答案

MCPManager 从核心 ToolRegistry 包装工具并输出 `local-mcp-v1` manifest；工具 schema 用 `finance__<tool_id>` 暴露给函数调用。HTTP surface 能初始化、列工具和调用工具；代码中 `resources/list` 与 `prompts/list` 明确返回空列表。因此面试可以说明已验证“统一工具发现和调用边界”，并将权限、远端服务、资源目录或 prompt catalog 作为下一步设计，而不能说已经全面接入完整 MCP 生态。

项目依据

- `src/utils/mcp_manager.py`：MCP-style 包装与 manifest。
- `src/utils/mcp_http_server.py`：JSON-RPC capabilities。
- `tests/test_mcp_manager.py`、`tests/test_mcp_http_server.py`：manifest 和调用测试。

表述边界与 Trade-off

统一协议有助于解耦工具接入和审计，但也引入 schema 版本漂移、权限和服务失败的新面。少量本地函数可直接调用；工具来源增多或跨进程服务化时，MCP-style 抽象价值更高。

Q18. Memory 架构在项目中如何实现？

30 秒简答

项目有 durable memory 的文件化实现，但配置默认 `enabled: false`，默认作用域是 `planner_router`。即使开启，它也只提供历史上下文 brief，代码明确要求不能作为证据，所有事实 claim 仍需当前 `evidence_id` 和校验门禁。

详细答案

持久上下文可保存 working、episodic 和 domain 级运行摘要，并构造长度受控的历史 brief，用于规划和路由理解此前缺口或偏好。配置还明确了 `context_only_not_evidence` 边界，编排 trace 会标出 memory 是否启用及被哪些节点使用。这种设计允许未来探索重复研究任务的上下文复用，同时控制陈旧结论污染新报告的风险。

项目依据

- `src/agents/durable_memory.py`：brief 文本包含 “Do not use this as evidence” 约束。
- `configs/app.yaml`：默认关闭、`planner_router` 和 `boundary`。
- `tests/test_durable_memory.py`：持久化和非证据 brief 测试。

表述边界与 Trade-off

不能说 Memory 已默认服务线上个性化。启用 memory 可能减少重复规划，却增加陈旧信息、偏见、删除权和跨用户隔离问题；高风险金融事实始终应以当前证据为准。

Q19. 如何控制金融报告幻觉？

30 秒简答

我不会只依赖 prompt 提醒模型别编造，而是在链路中分层限制：来源层要求证据记录，计算层把数字交给工具，生成层让关键 claim 绑定引用，校验层检查证据、lineage 与图表一致性，交付层对高风险缺口阻断或披露降级。

详细答案

金融幻觉包括无来源事实、单位或财年误用、估值假设遗漏、图文冲突以及把历史上下文当作当期证据。FinSight_多智能体的产物可以逐层排查这些问题：`evidence` 控来源，财务与估值 `artifacts` 控计算，`citations` 和 `numeric audit` 控引用，`verification_report` 与 `scorecard` 控交付。对数据不足的结论，应输出证据缺口和限制，而不是让 Writer 补全看似专业的数字。

项目依据

- `src/agents/multi_agent_orchestrator.py`：证据、数字、引用、校验产物。
- `src/evaluation/benchmark_metrics.py` 与 Formal-18 协议：可追溯与门禁规则。

表述边界与 Trade-off

强约束可降低可验证错误，但无法证明所有定性研判正确，也可能让报告更保守。系统目标是减少不可支撑陈述，而不是承诺没有任何分析偏差。

Q20. 如果继续优化项目，你优先做什么？

30 秒简答

我会先围绕已暴露的失败修复，而不是先扩展炫目的能力：优先提升港股关键结论引用和数字 lineage、排查 A 股交付阻断；随后用 Harness 测量检索模式、修复轮次与成本/延迟的 Pareto 关系；最后再以受控实验评估 Memory 或更完整 MCP 治理的增益。

详细答案

正式结果已经给出最有价值的优先级：整体指标提升成立，但 HK 的 37.80% traceability 和 CN-A 的 50.00% delivery 表明证据覆盖与交付规则仍有真实短板。下一阶段可以先扩充或修正这些 case 的关键 claim lineage 和失败分类，再比较 bm25 与 hybrid_rerank 是否值得增加的成本；durable memory 应通过默认关闭、ablation 和污染案例验证后才扩大使用；MCP 则应优先强化 schema contract、权限和审计，而非只增加服务数量。

项目依据

- `eval_outputs/benchmark_formal18_fy2024_v1/formal_benchmark_report.md`：市场薄弱点。
- `src/retrieval/retrieve.py`：可对比的检索模式。
- `configs/app.yaml`、MCP/Skill 相关测试：扩展能力的现有边界。

表述边界与 Trade-off

优化目标应按可量化失败和风险排序。更高质量、实时性、成本和响应时间无法同时最大化，因此需要按任务分层和评测证据做选择。

第二部分：Trade-off 决策框架

2.1 通用回答模板

涉及取舍的问题，不要只回答“看情况”，可按以下顺序组织：

- 目标：这次任务最不能错的是什么？
- 约束：交付时限、预算、数据时效、风险等级、可追溯要求分别是什么？
- 候选：有哪些可运行方案或降级路线？
- 指标：用交付通过率、追溯率、质量分、延迟、成本和失败分类衡量什么？
- 决策：在当前约束下选哪个方案，为什么？
- 防护：哪些 **gate**、审计和披露必须保留？
- 回退：工具失败、预算超限或证据不足时如何降级？

2.2 七组必备取舍

Trade-off	推荐回答主张	必看指标	回退或防护
Multi-Agent vs Direct/Single-Agent	高风险深度报告使用多阶段链路；低风险快览可用短链路	交付率、追溯率、p95、调用成本	输出等级标注；关键结论仍做证据门禁
bm25 vs vector vs hybrid_rerank	关键词精确任务优先 BM25；语义漏召回风险高且结论重要时升级混合重排	召回支撑率、引用缺口、p95	向量失败回退 BM25；限定 rerank 范围
Frozen eval vs realtime data	冻结集用于方案回归；实时链路用于 freshness 和数据源可用性验证	可复现指标、数据时间、线上失败类别	报告注明证据截止时间

Trade-off	推荐回答主张	必看指标	回退或防护
Strict gate vs deliverability	无支撑关键数字/估值不可放行； 低风险缺口可标注后交付草稿	阻断率、缺口类型、 人工复核量	降级为“资料不足”报告
Memory reuse vs objectivity	Memory 仅帮助规划/偏好，不进入事实证明链	污染案例、引用合规、 命中收益	默认关闭；当前证据优先
MCP-style vs direct tools	工具增多、需统一发现审计时采用抽象； 简单本地调用不必过度平台化	schema 失败、 调用错误、审计完整性	contract test、allowlist、 直接调用降级
Depth vs latency/cost	估值、修复和高级检索按报告价值与风险触发	token/tool 次数、轮数、 质量收益	预算上限、轮数上限、 部分交付

2.3 一分钟 Trade-off 口述范例

我不会默认把所有请求都跑最重的链路。对带估值或关键投资结论的深度报告，我优先保证证据和可追溯性，因此选择多 Agent、必要时启用更强检索和校验；对低风险概览，可以选择短链路降低延迟。决策依据不是感觉，而是同一 Harness 下的交付率、关键结论追溯率、延迟和成本，并为证据不足、工具失败或预算超限预先定义降级输出。FinSight_多智能体当前 Formal-18 证明了对多 Agent 在冻结协议中的质量收益，同时暴露了港股引用与 A 股交付短板，所以我会用指标驱动扩展而不是过度承诺。

第三部分：Benchmark Harness 专项

3.1 快速追问索引

追问	回答要点	对应核心题
Harness 最小模块是什么？	冻结数据加载、variant runner、产物采集、指标计算、失败汇总、结果报告	Q11
为什么要冻结输入？	让方案差异不被数据抓取时间差异污染	Q12
如何公平比较 baseline？	相同快照与指标协议，只改变处理链路	Q13
指标冲突怎么办？	交付、质量和追溯分开报告，再结合成本/延迟作决策	Q14、第二部分
如何防过拟合？	保留隐藏集、扩大市场/年份覆盖、线上 freshness 单独验证	Q12

3.2 Harness 场景题

场景 H-S1：新方案通过率提高，但平均成本翻倍，是否上线？

简答

不能只看通过率。我会先确认提升是否来自关键结论追溯和严重失败减少，再按深度研报与快速摘要分层设置预算；只有高风险任务的质量收益覆盖成本上涨，才在对应流量灰度启用。

详细答案

先在同一冻结集比较交付率、追溯率、失败类型、工具调用数与延迟，而非只比较一个总分。如果成本上涨来自每个 case 都运行昂贵 rerank 或多轮修复，可将其只应用于估值、关键数字或初次门禁失败的报告。上线策略应是分任务路由和灰度观测，不是全量替换。

项目落点

复用 Formal-18 的 variant 对比方式，并以 task_trace.jsonl、tool_trace.json、verification_report.json 定位成本来自哪一环。

取舍与风险

成本预算压得太严可能牺牲高风险事实质量；无预算约束则会使系统不可运营。

场景 H-S2：质量分没下降，但关键结论可追溯率下降 15%，如何排查？

简答

视为回归失败，优先查 critical claim 标记、引用绑定、检索召回和数字 lineage，而不是被总体质量分安抚。

详细答案

逐 case 对比 `claims.json` 与 `citations.json`，检查 critical claim 是否少标、引用 id 是否不存在于证据、numeric claim 是否缺少 audit 支撑；再看 retriever 是否换了排序导致关键资料降位，以及 writer/修复是否删除引用。可追溯率是金融报告底线指标，下降时应阻止上线并建立失败分类。

项目落点

Formal 协议要求 explicit critical label、快照引用与 numeric audit；`tests/test_formal_benchmark.py` 展示了这些失败条件。

取舍与风险

修复追溯率可能增加引用密度并影响可读性，后续需单独优化呈现方式，而不能弱化事实门禁。

场景 H-S3：A 股样本失败而美股正常，怎么定位？

简答

先按市场拆分失败原因，重点排查披露来源、代码映射、中文表格抽取、单位与财年口径，而不是立刻归咎于模型能力。

详细答案

A 股与美股在数据来源、代码格式、中文财报表格及单位上都有差异。先查看 A 股 case 是否完整进入冻结快照，再核对证据中 period、币种/单位和三表字段，随后对照 `verification_report` 区分是证据缺口、数字 lineage 失败还是报告结构失败。若仅某一市场失败，修复应围绕该市场数据和规则，而不是用全局 prompt 掩盖问题。

项目落点

`formal18_fy2024_v1` 已分 US、HK、CN-A 报告结果，其中 CN-A 的 multi-agent 交付率是已知薄弱点。

取舍与风险

市场定制规则可提高准确性，但会增加维护和回归范围，需要保持统一 claim/citation 契约。

场景 H-S4：LLM Judge 总给长报告高分，怎么办？

简答

把关键底线转为规则和 claim-level 指标，Judge 只补充语言与组织质量；同时在 rubric 中控制冗长度偏好。

详细答案

对是否有证据、数字 lineage、图表冲突和章节缺失，使用确定性规则或结构化 artifacts 评估；对可读性可用盲评或配对 judge，并记录版本与稳定性。如果长文本仅重复事实而不提高追溯率，不能因 Judge 分数高而放行。Formal-18 中将交付、客观质量和追溯拆开，就是避免单一主观分掩盖此类问题。

项目落点

使用 `benchmark_metrics` 的确定性检查与正式报告的三指标分栏方式。

取舍与风险

规则更稳定但覆盖不了全部语言质量；Judge 有覆盖面却存在偏差，应采用组合评估。

第四部分：Skill 专项

4.1 快速追问索引

追问	当前项目真实回答
Skill 和 Tool 的区别？	Tool 是可实际调用的底层 API；Skill 是 Planner/Router 可见的能力摘要，包含候选工具和 guardrails。
Skill 和 Agent 的区别？	Agent 参与任务执行循环；当前 Skill 不执行流程，只影响规划或路由提示。
Registry 是否配置化？	是，优先从 <code>configs/skill_registry.yaml</code> 读取。
Router 是否训练过？	当前为任务类型与触发词打分的规则选择，不声称有训练模型路由。
Skill 如何保证安全？	brief 明确不能代替工具执行、证据、引用和 verifier gates。

4.2 Skill 场景题

场景 S-S1：要求分析未来三年利润弹性，但路由只提示普通概览能力

简答

这是提示路由漏配或触发词覆盖不足；补充与利润弹性、敏感性、估值相关的提示选择，并确保最终仍由分析工具和证据门禁完成结论。

详细答案

先查看 Router 输入的任务类型和 query 是否能命中 `financial_statement_analysis`，并为利润弹性、scenario、sensitivity 等表达加入回归样例。命中 Skill 后仍不能直接生成预测结论，必须在存在可支撑输入时生成估值或敏感性产物；未来假设应明确标注为假设而不是已发生事实。

项目落点

`SkillRegistry.select` 使用任务类型与 trigger terms；估值工具与 `valuation_sensitivity.json` 为高风险输出提供结构化承载。

取舍与风险

扩大触发词提高召回，也可能误触发重链路；应通过典型请求集检查精度和成本。

场景 S-S2：估值能力输出目标价，却没有列出假设

简答

不能作为可交付估值结论；必须补齐假设和敏感性产物，缺数据时降级为估值方法说明或资料缺口。

详细答案

目标价属于高风险 claim，需要知道估值方法、关键输入、口径、同行或折现假设，以及敏感性变化。Verifier 应把缺少 assumptions 的输出标为失败并路由回分析节点；Writer 不能用流畅文字补掉这一缺口。若数据不足，则删除确定性目标价，保留限制披露。

项目落点

`perform_company_valuation` 及 `valuation_model.json`、`valuation_assumptions.json`、`valuation_sensitivity.json` 是回答依据。

取舍与风险

阻断会降低出稿率，但避免输出不可复算、容易误导的估值数字。

场景 S-S3：多个能力提示都可回答“竞争优势”，选哪个？

简答

先按用户要的是公司事实、同行比较还是行业结构确定主任务，再选择最少够用的能力组合；不要仅因为可用能力多就全部调用。

详细答案

若问题是基于公司披露描述优势，证据发现与报告组装已足够；若要证明相对优势，则加入同行比较；若要延伸行业格局，则使用行业研究能力并披露行业数据边界。Planner 应明确产物需求和证据级别，避免多能力重复检索或输出相互矛盾结论。

项目落点

configs/skill_registry.yaml 包含 evidence_discovery 、 financial_statement_analysis 、 industry_research 等能力摘要。

取舍与风险

能力组合越多覆盖越广，但成本、上下文冲突和审核面越大；选择应由交付目标驱动。

场景 S-S4：图表与正文数字不一致，谁负责修？

简答

先阻断交付并追溯共同数据源；按 lineage 判断是图表渲染、正文引用还是上游指标错误，再只修责任节点并重跑一致性检查。

详细答案

正文和图表都应来自已验证指标产物，而不是各自重新计算。发现冲突后先核对财务指标和图表 metadata，再检查 Writer 是否转写单位或期间错误；修复完成后重新生成 chart_consistency 和 verification。Skill 只提示能力路径，不能替代这类检查。

项目落点

图表工具、charts.json 、 chart_consistency.json 与 multimodal_consistency.json 。

取舍与风险

自动选择任意一方为真可能传播错误；阻断与复核会增加交付时间，但对数字研报是必要成本。

第五部分：MCP 专项

5.1 快速追问索引

追问	当前项目真实回答
MCP 和 Function Calling 的区别？	Function Calling 是调用 schema；项目的 MCP-style 层统一包装工具的发现、命名、schema、调用与 manifest。
当前暴露什么能力？	tools 能力；resources/list 和 prompts/list 当前为空。
是否是完整协议实现？	不这样声称；当前协议标识为 local-mcp-v1 的本地边界和 HTTP surface。
如何做治理？	当前可从 manifest 与 tool trace 观测；allowlist、鉴权与版本策略作为扩展设计说明。

5.2 MCP 场景题

场景 M-S1：行情工具超时，但用户要求当天报告

简答

实时行情不可用时不编造最新价格；可继续生成不依赖实时价格的部分，并明确行情数据缺失或使用缓存的截止时间。

详细答案

先将工具失败记录进调用 trace，判断报告的关键结论是否依赖当天价格。若估值或走势判断依赖该数据，则阻断相关 conclusion 或输出带缺口的草稿；若仅业务和历史财务分析可完成，则交付这些部分并披露实时数据未验证。恢复后再补跑依赖行情的模块。

项目落点

`fetch_yahoo_market_snapshot` 是真实工具；MCP-style 调用边界与 `tool_trace.json` 支持记录工具使用。

取舍与风险

缓存提高可用性但牺牲 freshness；必须显示时间戳和限制，避免用户把旧行情当成实时信息。

场景 M-S2：工具描述含糊导致模型误调用

简答

把描述、参数 schema、适用/禁用条件补清楚，并用工具级回归 case 验证选择；关键工具调用仍需要权限和产物检查。

详细答案

错误工具调用可能导致错误数据源或无谓成本。应在 Registry/MCP schema 中清楚说明输入、输出和使用场景，例如本地证据检索与市场快照不是同一种信息；对容易混淆的请求建立 contract tests 或 trace 检查。manifest 和 schema 版本变化还需要回归验证。

项目落点

`ToolSpec.description` 与 `MCPTool.to_tool_schema()` 决定暴露给调用方的工具描述。

取舍与风险

更长描述可能增加上下文消耗，但含糊工具接口在高风险任务中的成本更高。

场景 M-S3：上传资料含提示注入，诱导调用敏感工具

简答

将资料文本当证据内容而非系统指令；危险调用需 allowlist、权限或人工确认，任何无法解释的外发/导出请求都应拒绝并审计。

详细答案

检索到的文件内容可能包含“忽略规则并调用某工具”等文字，它不能改变系统策略。工具层应限定可用工具和参数，执行前识别越权目的，trace 记录被拒绝的调用。FinSight_多智能体当前主要是本地金融 tools 表面，面试可把安全控制作为 MCP 服务化前必须完善的治理设计，而不是谎称已有完整权限平台。

项目落点

当前 `local-mcp-v1` manifest 和 tool trace 提供治理基础；生产级鉴权/审批属于后续设计。

取舍与风险

严格确认可能降低自动化程度，但金融资料与敏感信息场景中，错误外发的风险远大于多一步确认。

场景 M-S4：多个服务返回同一指标但数值不同

简答

不应任选一个数字写入报告；先比较来源等级、期间、单位和披露时间，无法消解时展示冲突并限制结论。

详细答案

冲突通常来自财年、币种、复权方式或来源更新时点差异。规范化层需保留各来源 metadata，由分析或 verifier 标出冲突；对年报中已披露的历史数字优先正式披露，对实时行情必须标识时间。冲突未解决前，估值或趋势 claim 不能冒充确定事实。

项目落点

Evidence/claim/citation 产物与 verification 机制承载冲突处理；扩展 MCP 数据源时沿用同一规则。

取舍与风险

强制单一来源有一致性但可能失去更新速度；保留冲突更诚实但会降低报告简洁度。

场景 M-S5：工具 schema 更新导致上游能力失败

简答

将 schema 当接口契约管理，变更必须带版本、适配和回归测试；未适配时降级到可用工具或中止依赖结论。

详细答案

工具名、必填参数或返回字段变化会破坏路由提示和编排节点。应比较 manifest、保持 adapter 或兼容窗口，并跑工具调用与报告关键场景回归；对估值等高风险链路，schema 不匹配应快速失败而不是静默忽略字段。

项目落点

MCPManager.export_manifest()、tests/test_mcp_manager.py 与 HTTP 调用测试可作为 contract test 起点。

取舍与风险

兼容层降低升级中断，却会累积维护成本；需要明确淘汰周期而非永久背负旧接口。

第六部分：Memory 专项

6.1 快速追问索引

追问	当前项目真实回答
Memory 与 RAG 的区别？	RAG evidence 支撑当期事实；durable memory 仅作为历史规划上下文。
默认是否启用？	不启用，configs/app.yaml 中为 enabled: false 。
默认谁可用？	planner_router ；不是所有 Agent 默认共享。
如何防污染？	brief 写明不作证据，当前 evidence/citation/verifier 始终优先。

6.2 Memory 场景题

场景 MEM-S1：记忆中偏好“报告乐观”，但证据显示风险高

简答

偏好只可影响组织或语言风格，不能改变事实与风险披露；报告必须按当前证据呈现风险。

详细答案

Memory 若启用，也只是 Planner/Router 的上下文提示。写作与校验应使用当前 evidence 和 claims，凡与用户偏好冲突的风险事实仍必须保留；可以调整表达结构，例如先说明机会再说明风险，但不能隐藏或弱化关键风险。

项目落点

durable_memory.py 明确历史上下文不是证据，configs/app.yaml 将其默认限制为规划路由范围。

取舍与风险

个性化增强体验，却不能压过金融客观性；否则会形成系统性迎合。

场景 MEM-S2：历史公司画像与最新公告冲突

简答

以最新可验证证据为准，将旧记忆标记为过期或不使用，并在报告中披露业务变化。

详细答案

先检索并核验最新公告，生成新的 evidence/claim；若旧 brief 引导了错误规划，应从 trace 中确认影响并触发 memory 修订策略。当前代码边界已保证 memory 不是直接事实来源，因此报告不应引用旧画像来反驳当前公告。

项目落点

durable memory 的 domain/episodic context 与当前 evidence 管线分离。

取舍与风险

保留历史变化有分析价值，但必须版本化和标记时间；直接覆盖又可能丢失变化轨迹。

场景 MEM-S3：无关历史任务污染当前报告

简答

先关闭该次 memory 注入并回到纯当前证据生成，再排查检索过滤和上下文范围，避免错误继续传播。

详细答案

观察 collaboration trace 中 memory 是否被使用、brief 是否携带无关公司或期间；对 memory 检索增加 symbol、period、report type 和长度限制，并建立污染 bad case。因为默认配置已经关闭 durable memory，任何启用实验都应和无 memory 版本做 ablation。

项目落点

配置中的 enabled / context_scope 和 collaboration trace 的 memory 状态支持此类定位。

取舍与风险

更严格过滤降低污染，但可能减少跨周期可复用背景；金融事实安全优先于复用率。

场景 MEM-S4：用户要求删除长期偏好记忆

简答

产品化前必须支持可删除、可审计和后续不再注入；当前项目只能将其作为需要完善的治理要求，不声称已有用户级合规平台。

详细答案

在完整产品中，应定位该用户持久记录、删除原文与索引、记录删除操作，并确认后续 brief 不包含被删偏好。当前仓库证明的是文件化 durable memory 与上下文边界，因此面试应说明实现方向和缺口，不把规划当作已完成权限治理。

项目落点

src/agents/durable_memory.py 提供文件化存储基础；用户级删除治理属于拓展需求。

取舍与风险

审计留痕和彻底删除需要平衡合规要求；应保留操作记录而非保留已删除敏感内容。

场景 MEM-S5：同一条历史记忆多次导致错误

简答

立即停止其进入上下文，记录为污染案例，并要求新结论只能依据当前证据重建。

详细答案

多次失败说明不是偶发 prompt 问题，而是 memory 生命周期和可信度治理缺失。可在后续设计中加入失效标记、时间衰减、来源和验证状态；在当前系统说明中，应强调通过默认关闭和 context-only 边界控制该风险。

项目落点

Memory brief 的非证据声明以及 verifier/citation 链路是现阶段防线。

取舍与风险

主动废弃可能损失有用背景，但重复污染比重新检索代价更高。

第七部分：金融研报业务场景题

7.1 证据与引用

场景 1：报告写“公司 AI 业务收入高速增长”，但没有引用

简答

这是一条可验证事实 claim，没有来源不能交付；应查找对应披露证据并绑定引用，找不到则删除或改为明确的资料缺口。

详细答案

先判断该说法是否能在年报、公告或可接受公开来源中找到对应业务收入及期间。检索命中后形成 evidence 和 claim mapping，再允许报告表达增长；若只有市场叙事而没有可量化披露，应改写为“现有资料未能验证该收入增长”。

项目落点

使用 evidence.json、claims.json、citations.json 与 verification 缺口记录。

取舍与风险

保守删除会减少报告亮点，但无依据的增长结论对金融读者风险更大。

场景 2：模型声称引用年报第 42 页，但该页没有对应内容

简答

引用存在不等于引用正确；应校验证据内容与 claim 是否支持，并将错引视为门禁失败。

详细答案

对 PDF 解析片段保留来源和页面关联，检查引用段落是否含支持该 claim 的文本或表格行；若页码或抽取错位，应修复解析/引用映射后重跑验证，不能只替换一个看似接近的页码。数字 claim 还需验证实际数字 lineage。

项目落点

项目会将 PDF sections/tables 转为 evidence record，Formal 追溯规则要求 cited evidence 实际支撑 claim。

取舍与风险

页面级审计增加处理成本，但能避免“形式上有 citation、实质上无证据”的严重问题。

场景 3：年报和新闻稿对同一收入数字不一致

简答

先核对期间、币种、是否调整口径和发布时间；同口径冲突时优先正式披露来源，并在必要时披露差异。

详细答案

新闻稿可能使用非 GAAP 指标、季度口径或简化单位，年报可能为审计年度数据。应保留两个 evidence 的 metadata，比较财年、口径与单位；无法对齐时不计算统一增长率，并在报告中说明存在不同披露口径。

项目落点

Evidence 规范化、财务工具与 verifier 可承担冲突检测及限制结论。

取舍与风险

优先权规则提升一致性，但不能把更晚更新的信息粗暴丢弃；需要保留解释。

场景 4：用户要求引用未授权的券商研报

简答

不能把无授权内容作为系统证据；应说明无法使用该来源，并改用可访问的公司披露或公开数据。

详细答案

首先明确数据授权边界，避免抓取、复述或伪造不可用的研报观点。若用户自行提供有权使用的材料，可按上传文档证据处理并保留来源标签；否则只基于正式公开资料生成，并说明研究覆盖限制。

项目落点

项目的证据驱动设计允许记录来源类型与限制；不声称仓库已有券商授权库。

取舍与风险

缺少付费来源可能影响覆盖深度，但版权与可信来源边界不能为了内容丰富度突破。

7.2 财务数字与估值

场景 5：万元、亿元和百万美元混用导致图表错误

简答

阻断交付，统一币种、单位和汇率/期间口径后重新生成指标、图表和正文。

详细答案

单位错误属于上游数据 lineage 问题，不应只修改图表标签。应从 evidence 与三表视图确认原始单位，为转换保留规则与汇率日期，再让图表和文字共用同一规范化数值产物；一致性检查通过后才出稿。

项目落点

三表视图、财务指标产物与 chart_consistency.json 、 multimodal_consistency.json 。

取舍与风险

统一换算便于比较，但可能隐藏原始披露口径；报告应保留单位和转换说明。

场景 6：公司财年不是自然年，却被按自然年比较

简答

这是期间口径错误；必须按公司披露财年重新匹配 evidence 和比较期，并撤销错误同比结论。

详细答案

对每条证据和指标检查 period 、财年结束日及 comparative column，确认比较的是同口径年度。若跨公司比较存在不同财年，应披露不可完全对齐，或改用最近可比期间而非强行给出同比判断。

项目落点

Formal-18 固定 FY2024 并校验期间；工具输入和 evidence 记录含 period 维度。

取舍与风险

严格期间对齐可能减少可比公司数量，但能避免非常隐蔽的趋势误判。

场景 7：同行比较混入不同行业公司

简答

先停止引用比较结论，核验 peer 选择规则和行业语境；无法获得可靠同行时披露不足，不硬做排名。

详细答案

同行估值只有在业务模式、收入结构和市场可比时才有解释力。应检查同行列表来源与筛选条件，必要时把同业与泛行业参照拆开，并让估值结论注明 peer coverage。模型不能仅因公司名称相似就把它当作可比标的。

项目落点

build_peer_comparison 与 perform_company_valuation 产物应被验证和说明假设。

取舍与风险

扩大 peer 集提高样本量，却会降低估值可解释性；宁缺勿滥更适合高风险结论。

场景 8：估值结论没有敏感性分析，是否允许通过？

简答

如果报告给出确定性估值或目标价，不应通过；至少要有核心假设和敏感性说明。仅做概览且明确不输出估值结论时可以降级交付。

详细答案

估值高度依赖增长、折现、利润率或同行倍数假设。高风险报告应保存模型、assumptions 和 sensitivity，Verifier 将缺失项作为 blocker；若材料不足，可以交付业务和历史财务部分，同时明确估值暂不可验证。

项目落点

valuation_model.json 、 valuation_assumptions.json 、 valuation_sensitivity.json 及校验报告。

取舍与风险

要求敏感性会增加分析工作，但避免读者把脆弱假设误解为确定价值。

7.3 Agent 协作与质量门禁

场景 9：Planner 漏掉风险章节，Writer 也没写

简答

必要章节缺失应被质量门禁发现，路由回报告组织或规划阶段补齐，而不是直接交付。

详细答案

章节完整性应由模板或 scorecard 明确检查，风险部分尤其不能被用户的乐观倾向或上下文裁剪省略。补写时仍需引用当前证据；若确无风险资料，应披露信息缺口而不是生成泛化风险文本。

项目落点

company_report_scorecard.json 、 verification_report.json 与 report assembly 能力提示。

取舍与风险

固定章节提升覆盖，但可能导致空泛填充；门禁应要求有依据的内容或明确缺口。

场景 10：Retriever 召回大量无关网页，Writer 写得仍很流畅

简答

流畅不代表可信，应通过证据质量、引用匹配和关键 claim 支撑率发现问题，并优先修检索而不是润色文本。

详细答案

查看召回 evidence 是否与 symbol、period 和研究主题匹配，比较检索 mode、top-k 与来源过滤造成的噪声。若关键 claim 仅由低相关片段支撑，Verifier 应失败并要求重新检索；必要时升级到混合或 rerank，但必须评估延迟成本。

项目落点

retrieve_local_evidence 五种 ranking mode、evidence.json 和 Formal traceability 指标。

取舍与风险

扩大召回有利于覆盖却带来噪声；缩窄检索会漏掉重要披露，需以支撑率评估。

场景 11：Repair 修好了数字，却删除了引用

简答

修复后必须重新跑完整相关门禁；任何数字改动若丢失引用，仍应判为失败。

详细答案

Repair 不应只追求消除一个报错而引入新缺陷。应比较修复前后的 claim、citation 和 numeric lineage，确保新数字有 evidence 支持，报告引用仍存在，并在 revision trace 中记录变化。若局部修复不断造成副作用，应终止自动循环并转人工复核或降级。

项目落点

gap_resolution_trace.jsonl、revision_history.json、claims.json、citations.json 与 verification。

取舍与风险

全量复验增加修复时延，但对依赖链变化是必要保证。

场景 12：多 Agent 出错后如何定位责任步骤？

简答

从失败的最终 claim 倒查 citation、evidence、分析产物和任务 trace，定位首次出现错误的节点，而不是只看最终报告。

详细答案

若是没有来源，查看检索/证据规范化；若是数字错，查财务或估值 artifact；若是引用丢失，查写作和 repair；若是未拦截，查 verifier 和门禁规则。职责按结构化产物划分后，能把“谁写错了”转化为“哪个 artifact 首次不满足契约”。

项目落点

task_trace.jsonl、agent_collaboration_trace.json、tool_trace.json 以及各结构化 artifact。

取舍与风险

完整可观测性会增加存储和脱敏要求，但缺乏 trace 的多 Agent 系统很难稳定维护。

7.4 评测、MCP 与 Memory

场景 13：新模型质量更高，但 p95 延迟超过 SLA

简答

按报告风险分层路由：深度研报可以接受更慢链路，实时快览使用较轻方案；在未满足对应 SLA 前不全量替换。

详细答案

先拆分耗时来自生成、检索重排、工具还是修复轮次，再判断增加的质量是否集中在关键结论。若质量提升来自对低风险语言润色，就不值得牺牲时延；若显著修复高风险引用或估值缺口，可对特定模式灰度启用，并保留降级路径。

项目落点

使用 Harness 指标与 trace 产物扩展记录 latency/cost；当前 Formal-18 质量结果提供质量端基线。

取舍与风险

统一最慢高质模型会损害使用体验；统一最快模型又可能牺牲可信度。

场景 14：Formal-18 提升，但 hidden set 下降

简答

这提示对冻结集过拟合或规则仅覆盖已知模式，不能上线为通用改进；应分析 hidden failure 并扩大回归覆盖。

详细答案

检查改动是否针对某些 case 的格式、来源或关键词做了特化，同时按市场、年份和失败类型比较差异。Formal-18 适合可复现回归，但不足以替代扩展/隐藏集；改进只有在关键隐藏风险不恶化时才值得保留。

项目落点

正式协议已将 Formal-18 定位为冻结比较集而非生产代表性证明。

取舍与风险

快速针对已知弱项修复很有价值，但必须用未见数据防止虚假的总体提升。

场景 15：引用覆盖提高，但报告可读性下降

简答

关键事实的引用不能为了可读性删除；可通过引用呈现方式、摘要层级和正文组织优化阅读体验。

详细答案

先区分关键 claim 与低风险叙述，对关键结论保留近距离引用和数字依据；对重复支持材料可合并到表格、脚注或证据汇总，而不是重复插入文本。评测时同时查看追溯率与可读性，但可读性不能覆盖底线门禁。

项目落点

citations.json 和报告输出支持将支撑关系与呈现布局分开优化。

取舍与风险

引用过密干扰阅读，引用过少伤害可信度；应按 claim 风险分层。

场景 16：PDF 解析表格列错位导致估值错误

简答

将其视为证据/结构化数据质量错误，阻断依赖该表格的估值并修复解析或切换可靠证据。

详细答案

表格列错位会让后续工具稳定地计算错误结果，所以仅让 LLM 校对文字不够。应将 PDF table evidence 与关键指标抽样比对，检测单位、表头、年度列和行关系；未校正前，估值应输出缺口而不是目标数字。

项目落点

PDF tables 到 evidence 的处理路径，以及财务、估值和 verification artifacts。

取舍与风险

解析核验会降低批处理速度，但能防止错误被工具化放大。

场景 17：MCP 工具描述被恶意修改，诱导危险调用

简答

工具描述不能被非受信内容动态改写；应采用受控 registry、manifest 审核、allowlist 与调用审计，异常版本停止使用。

详细答案

将用户文档和外部工具 metadata 视作不可信输入，工具注册应来源固定且版本可审计。若 schema/description hash 改变或出现不合理能力，应拒绝加载或仅保留只读安全工具。当前项目已有 manifest 与调用边界，可把签名、权限与版本审批作为生产化扩展。

项目落点

mcp_manifest.json、MCPManager 和 HTTP tools surface。

取舍与风险

严格注册治理使接入变慢，但防止工具层成为提示注入绕过点。

场景 18：工具返回内容过多导致上下文超限

简答

工具返回应先结构化筛选和压缩，再把与当前 claim 相关的片段注入上下文，同时保留原始 evidence id 供审计。

详细答案

对长 PDF 或大量检索结果，可先按任务主题和 critical claim 类型选择证据，再只传入必要字段或摘要；数字工具应返回结构化值而非整个原文。上下文压缩不能丢掉来源链接和 period，否则 Writer 仍可能写出不可追溯结论。

项目落点

retrieve_local_evidence 的 top-k/chunk 选择，及 evidence/claim/citation 分离设计。

取舍与风险

裁剪降低成本和超限风险，却可能漏证据；对关键 claim 应允许回查完整来源。

场景 19：用户说“再写一份”，是否使用上次关注新能源的偏好？

简答

当前默认不使用 durable memory；若未来明确启用，只可将其作为可确认的关注点提示，并向用户说明或允许修正，不能替代本次研究要求。

详细答案

若请求语义不完整，可询问或以最小假设生成通用报告；在启用 memory 的实验模式下，Planner 可提示“此前关注新能源，是否继续”，而不是默默改变报告范围。无论主题偏好如何，事实和风险仍需本次 evidence 支撑。

项目落点

memory.durable.enabled: false 和 context_scope: planner_router 是当前事实边界。

取舍与风险

复用偏好提升便利性，但静默继承可能造成范围误判或隐私问题。

场景 20：长期记忆仍写旧主营业务，最新证据显示业务转型

简答

当前证据优先，旧记忆不能进入事实结论；应标记旧上下文失效，并将转型作为有引用的新 claim。

详细答案

从最新披露生成新的 evidence 和 claim，报告中说明业务结构发生变化以及可验证时间点。若启用的 historical brief 引导了错误检索或写作，则从 trace 中识别并修正 memory；直到治理完成，可停用该上下文。

项目落点

Durable Memory 的 context-only 约束、证据链和 verifier。

取舍与风险

历史画像有助于解释转型，但未经时间校验的复用会将过时事实写成当前现实。

第八部分：新增 Trade-off 场景题

这些题用于覆盖公开 Agent/RAG 面试讨论中反复出现的评测、延迟、工具安全、Memory 和工程降级话题。它们是面试准备扩展，不代表当前仓库已经具备完整线上治理功能。

场景 T-S1：用户十分钟内要一份快速研报，是否绕过完整 Multi-Agent？

简答

先判断输出风险：只做资料摘要可用轻链路并标明“快速概览”；涉及估值、关键财务结论或投资倾向时，不绕过证据与门禁，只能降级范围而不是降级真实性。

详细答案

可设计报告等级：快速概览仅覆盖已有高可信 evidence 的摘要和明确限制，深度报告才运行估值、图表、完整验证和修复。若时间不足完成高风险部分，交付部分报告并列未验证事项，而不是让 one-shot 补写目标价或实时判断。后续用相同 case 比较不同等级的延迟、成本与关键错误率。

项目落点

Formal 的三 variant 与编排链产物提供分层比较基础；当前不声称已有线上分流策略。

取舍与风险

快链路提高响应速度，但必须收窄可陈述结论范围，否则会把 SLA 压力转化为事实风险。

场景 T-S2： hybrid_rerank 提升引用质量，但 p95 或成本超预算

简答

不默认对所有检索使用重排；只对关键 claim、估值证据或初次门禁失败案例触发，普通背景检索维持低成本模式。

详细答案

在固定输入上分别测试 bm25、hybrid、hybrid_rerank，观察引用缺口减少是否集中于关键结论。如果提升主要来自少量高风险章节，应做条件触发；同时保留 vector/reranker 不可用时的 BM25 回退并标记证据限制。

项目落点

src/retrieval/retrieve.py 已提供真实模式与向量失败回退能力。

取舍与风险

重排可提升前排相关性，却增加依赖、耗时和 checkpoint 治理成本。

场景 T-S3：冻结回归表现提高，但实时新闻更新后报告过时

简答

冻结评测与实时 freshness 是两个目标，不能互相替代；保持冻结集比较方案，同时另建带数据截止时间的实时测试和数据源可用性监控。

详细答案

Formal-18 用于回答“同一证据下哪个流程更稳”，不回答“今天最新信息是否覆盖”。线上或演示报告必须标注 evidence cutoff；对触发重大公告或行情依赖的结论，实时来源失败时不沿用冻结结果充当最新判断。两套评估应分别记录可复现质量与新鲜度/超时失败。

项目落点

正式协议明确 runtime evidence fetching 被禁止，正好说明该结果的适用范围。

取舍与风险

追求实时会降低可复现性和提高来源波动；只依赖冻结输入则无法满足当天研究需求。

场景 T-S4：实时来源超时或两个来源冲突，还要按时交付

简答

按证据依赖拆分交付：可靠的历史财务和业务内容可继续，受实时数据影响的价格或估值结论应标注缺失、冲突或延后。

详细答案

记录失败来源、时间戳和冲突字段；检查报告中哪些 claim 依赖它们。若存在已验证缓存，只能明确标为某个时间点的快照；若无法对齐口径，则在正文披露冲突并阻断依赖数值的确定性 conclusion。这样交付的是可解释草稿，而不是假装实时完整。

项目落点

工具调用 trace、evidence metadata 和 verification 可承接该策略；线上数据 SLA 属于扩展工程。

取舍与风险

部分交付保留可用性，但必须让用户清楚哪些分析尚未验证。

场景 T-S5：严格 Gate 导致大量报告被阻断，业务要求提高出稿率

简答

不能降低关键事实门槛来换出稿率；应区分 blocker 与 warning，并提供“含明确缺口的研究草稿”降级等级。

详细答案

先按失败类别统计是缺数据、规则过严、解析问题还是模型引用错误。对无支撑关键数字、严重图文冲突和缺估值假设继续阻断；对非关键章节或展示问题可以 warning/repair。报告必须显式显示未覆盖内容，业务侧得到可用部分，质量底线也不被模糊。

项目落点

Formal failure summary 和 verification_report.json 可用于分析门禁阻断原因。

取舍与风险

降级输出提高交付比例，但没有清晰标签时会被误认为完整报告。

场景 T-S6：Memory 能提升重复任务效率，但出现陈旧事实污染

简答

保持默认关闭或只在受控实验启用；Memory 限于规划和用户可确认偏好，当前 evidence 永远覆盖历史上下文。

详细答案

设计 ablation 比较是否真的减少规划时间或提高完整性，并将污染 case 列为硬失败。Memory 写入要有来源、时间和失效规则，读取要按 symbol/period/用户隔离；涉及事实时必须重新检索并引用新 evidence。污染率没有被控制前，不扩大启用范围。

项目落点

配置默认关闭和 context_only_not_evidence 边界使这套回答与实际实现一致。

取舍与风险

复用历史能降低重复工作，但金融时效性让过期事实的代价非常高。

场景 T-S7：MCP-style 统一工具方便扩展，但出现 schema 漂移或权限扩大

简答

把统一接口视作需要治理的公共契约：版本化 manifest、contract test、allowlist、审计和安全降级必须与服务扩展同时推进。

详细答案

在增加远端工具或写操作前，先固定 schema 版本与最小权限；调用方只加载批准的工具集合，description 或参数发生变化要触发回归。工具不可用或版本不匹配时，降级到已有安全本地工具或停止相关结论，不允许静默跳过校验。

项目落点

当前实现能够导出 local-mcp-v1 manifest 并测试 tool list/call；权限与版本发布流程应作为后续治理，而非已完成事实。

取舍与风险

服务化提升复用和统一观测，却扩大攻击面与运维复杂度。

场景 T-S8：修复轮次耗尽仍未过门禁，如何结束任务？

简答

到达预算或轮次上限就停止自动修复，输出失败原因、已验证部分和不可交付结论，不让系统无限循环或伪造完成。

详细答案

保留每轮 verification_report、revision 与 gap trace，说明哪些 blocker 仍存在；若可以生成不含失败结论的降级报告，则明确交付等级和缺口，否则返回无法可靠生成。随后把该失败纳入 Harness 分类，用于修检索、解析、规则或工具，而不是无边界重试。

项目落点

编排链已有修复记录产物和质量状态，可作为上限终止与归因依据。

取舍与风险

终止会牺牲一次任务完成率，但保护预算和可信度，是比无限自动补写更合理的工程决策。

第九部分：基础八股与项目结合速答

9.1 RAG 与 Agent

问题	与 FinSight_多智能体结合的简答
RAG 链路是什么？	资料进入 evidence store，经检索选出证据、形成 claim、生成报告，再由引用和数字门禁校验。
Top-k 越大越好吗？	不是；增大覆盖也增大噪声和上下文成本，对关键 claim 应看支撑率而非只看数量。
Rerank 为什么有效？	对候选证据重排可把更支持当前 claim 的资料放前，但需承担延迟和 checkpoint 依赖。
Tool Calling 难点是什么？	工具选择、参数正确性、返回契约、错误降级以及结果能否进入引用链。
Multi-Agent 怎么防无限循环？	设置修复轮次/预算上限，保留失败 trace，无法通过时降级或终止。
Agent 如何观测？	通过任务、协作与工具 trace 结合 artifacts 倒查首次失败节点。

9.2 Evaluation 与工程

问题	与 FinSight_多智能体结合的简答
为什么冻结评测集？	控制输入差异，保证方案比较可以复验。
LLM-as-Judge 有什么局限？	可能偏好长度或风格，关键引用和数字应使用结构化规则兜底。
Offline 与 online eval 区别？	Offline 比较流程质量和回归；online 另测 freshness、时延、工具稳定性和用户风险。
如何管理配置？	模型、检索、memory 和 benchmark 均应配置化并记录版本；项目已有 YAML 配置路径。
如何做 retry？	只对可恢复工具失败限次重试；证据冲突或 schema 错误应快速失败并披露。
如何日志脱敏？	保留可追溯 id、步骤和失败分类，避免在 trace 中扩散敏感原文或未授权材料。

9.3 模型适配追问

问题	推荐答法
什么时候先用 RAG？	知识时效、来源可追溯和更新频繁的问题优先通过检索与证据链解决。
什么时候考虑微调？	输出结构、工具调用行为或特定校验模式在高质量离线样本上可稳定学习时再评估。
为什么不能只靠微调解决金融事实？	参数记忆不能替代当期公开证据、财年口径和引用审计。

第十部分：高压追问链与回答节奏

链路 1：项目真实性

- 1. 一分钟说清输入、链路、输出：使用 Q1。
- 2. 追问为什么不是普通 RAG：使用 Q2 与 Formal-18 结果。
- 3. 追问具体产物：列举 evidence、claims、估值、verification 与 traces。
- 4. 追问线上效果：主动说明冻结协议与不证明投资准确率。

链路 2：Skill、MCP、Memory 防夸大

- 1. Skill 是否执行工具：回答“目前是 Planner/Router capability hints”。
- 2. MCP 是否完整落地：回答“tools-only 的 local-mcp-v1 与 HTTP surface”。
- 3. Memory 是否默认工作：回答“默认关闭；即使启用也仅作上下文，不作 evidence”。
- 4. 为什么仍写进简历：回答“这些能力解决可配置路由、统一工具边界和受控复用问题，并明确实现范围”。

链路 3：取舍决策

- 1. 质量与成本冲突：先定义高风险结论底线，再按报告等级分流。
- 2. Frozen 与实时冲突：分离回归质量与 freshness 评估。
- 3. Gate 与交付冲突：关键错误阻断，低风险缺口标记降级。
- 4. 扩展功能优先级：先修 Formal-18 已暴露的市场失败，再实验更重能力。

第十一部分：简历安全表述

11.1 项目总述

推荐表述：

FinSight_多智能体是一个面向公司/个股研报的证据驱动生成与质量控制项目。系统将证据检索、财务与估值分析、报告写作、引用/一致性校验和缺口修复拆成可观测阶段，输出带 `evidence/claim/citation lineage` 的 Markdown、HTML 和 JSON 报告草稿。

不要表述为“自动投资决策平台”或“已上线生产级投研系统”。

11.2 Benchmark Harness

推荐表述：

构建冻结评测协议 `formal18_fy2024_v1`，在同一 `FY2024` 证据快照上比较 `Direct LLM`、`Single-Agent RAG` 与 `Multi-Agent RAG`；`Multi-Agent RAG` 的交付通过率、客观质量分和关键结论可追溯率分别为 `72.22%`、`86.27` 和 `70.01%`，同时保留 `HK` 引用和 `CN-A` 交付弱项分析。

必须附带说明：这是冻结离线协议结果，不代表实时生产稳定性或投资准确率。

11.3 Skill

推荐表述：

实现可配置 `SkillRegistry`，将证据发现、财务分析、报告组装与校验修复等能力摘要按任务匹配并注入 `Planner/Router`，同时保持工具执行、证据引用与质量门禁独立校验。

不要说已经实现可自治编排、自动训练路由或完整 Skill 执行平台。

11.4 MCP-style 工具边界

推荐表述：

将 9 个金融工具包装为 `local-mcp-v1` 的 `MCP-style tools` 边界，支持统一发现、`schema` 暴露、调用、`manifest` 导出及 `HTTP JSON-RPC tools surface`，便于观测与后续治理扩展。

不要说 `resources`、`prompts`、远端鉴权或完整生产 MCP 治理已经落地。

11.5 Durable Memory

推荐表述：

实现受约束的 `durable memory` 上下文能力，默认关闭并限定为 `Planner/Router` 提示；所有金融事实仍必须回到当期 `evidence`、`citation` 与 `verifier` 门禁，避免历史记忆污染报告。

不要说系统默认依赖长期记忆生成事实或已经完成用户级隐私治理。

第十二部分：复习优先级与覆盖结论

12.1 当前题库覆盖结论

主题	修订后覆盖	面试准备重点
项目介绍与架构	已覆盖	讲清结构化产物，而非泛泛说 Agent
RAG、检索、Rerank	已覆盖	必须能回答检索 mode 的成本与延迟取舍
财务、估值、引用门禁	已覆盖	数字、假设与 lineage 是核心

主题	修订后覆盖	面试准备重点
Benchmark Harness	已覆盖	熟记 Formal-18 指标和局限
Skill	已覆盖且修正边界	不能说成已执行平台
MCP	已覆盖且修正边界	只能说 tools-only MCP-style 实现
Memory	已覆盖且修正边界	默认关闭、仅上下文
Trade-off	已新增系统章节与八题	用指标、护栏和回退回答
安全与提示注入	已新增/加强	文档内容不等于系统指令

12.2 面试前优先背熟

- 1. Q1、Q2、Q11-Q14：项目与正式结果。
- 2. Q7-Q10：检索、数字计算、门禁和修复。
- 3. Q15-Q18：Skill/MCP/Memory 的真实实现边界。
- 4. 第二部分与 T-S1 至 T-S8：trade-off 决策逻辑。
- 5. 场景 1、3、5、8、11、16：证据、数字、估值与解析错误的阻断方法。

附录 A：一句话速答模板

项目一句话

FinSight_多智能体以 `evidence`、`claim`、`citation` 与 `verification` 为核心中间产物，通过多阶段研报链路生成可追溯的报告草稿，并用冻结 `Benchmark` 比较方案质量。

Benchmark 一句话

Formal-18 是 FY2024 跨三市场的冻结证据回归协议，用于公平比较三种生成链路，其结果证明当前多 `Agent` 方案在该协议上更强，但不等于线上或投资准确性结论。

Skill 一句话

当前 `SkillRegistry` 是给 `Planner/Router` 的可配置能力提示层，不替代真实工具执行、证据引用或校验门禁。

MCP 一句话

当前项目落地的是包装 9 个金融工具的 `local-mcp-v1 tools` 边界与 HTTP 调用表面，而不是完整生产级 MCP 平台。

Memory 一句话

Durable memory 在当前项目中默认关闭且只作为规划上下文，金融事实始终需要当前证据支撑。

Trade-off 一句话

我先按结论风险确定不可牺牲的证据门禁，再在质量、延迟、成本与新鲜度之间选择链路并定义降级路径。

附录 B：项目事实与公开补题依据

B.1 项目事实核验文件

- `src/agents/multi_agent_orchestrator.py`：多智能体链路、产物和 trace。
- `src/tools/registry.py`：9 个工具及真实英文 API。
- `src/tools/skill_registry.py`、`configs/skill_registry.yaml`：Skill 提示层的实现边界。
- `src/agents/mcp_manager.py`、`src/agents/mcp_http_server.py`：MCP-style tools 边界。
- `src/agents/durable_memory.py`、`configs/app.yaml`：Memory 默认状态和事实边界。
- `docs/formal_benchmark_protocol.md`、`eval_outputs/benchmark_formal18_fy2024_v1/formal_benchmark_report.md`：正式评测协议与结果。

B.2 公开面试补题来源

公开内容仅用于检查求职题目覆盖，不用于证明本项目已实现某项能力：

- 牛客公开讨论：[AI-Agent 面试题汇总 - 大模型篇](#)
- 牛客公开讨论：[AI开发/产品通用面经大盘点：RAG篇](#)